

Agentic RL全流程技术分析与总结（两万字）



笨鸟先飞
持续学习~

已关注

收录于 · Agentic技术 >

HYY CS、suc16 等 447 人赞同了该文章 >

在当前人工智能领域，尤其是强化学习（Reinforcement Learning）持续演进的背景下，学界对强化学习基础算法已有大量系统性研究。然而，关于强化学习如何赋能智能体

（t）、并推动其在实际场景中落地应用的综合性介绍与实战案例解析仍相对有限。本文从强化学习技术出发，探讨其如何驱动智能体在复杂环境中学习和决策，从而真正将大模型的决策能力转化为现实世界中的生产力，顺便总结下这段时间的工作。

一、RL基础知识

1、什么是RL

强化学习是机器学习的三大核心分支之一（另外两个是监督学习、无监督学习），核心逻辑是：智能体（Agent）通过与环境（Environment）的持续交互，通过“试错”学习最优行为策略，以最大化长期累积奖励（Reward），其流程图如下所示：

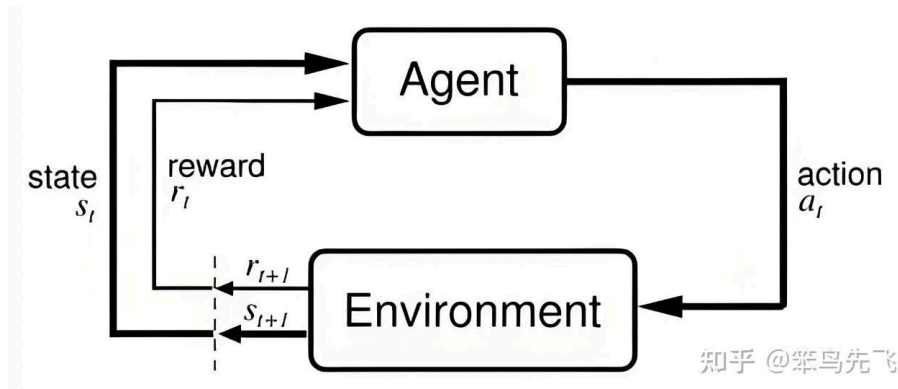


图1. RL基本流程

可以用一个通俗的类比理解：

- 智能体（Agent）= 正在学习的“机器人 / 模型”（比如自动驾驶汽车、大模型、工业机械臂）；
- 环境（Environment）= 智能体所处的场景（比如城市道路、对话场景、工厂生产线）；
- 状态（State）= 环境的实时情况（比如道路拥堵、用户的提问、机械臂的位置）；
- 动作（Action）= 智能体的决策（比如刹车、模型的回复、机械臂的抓取动作）；
- 奖励（Reward）= 环境对动作的反馈（比如安全通过路口得正奖励、用户满意回复得正奖励、抓取失败得负奖励）；
- 策略（Policy）= 智能体学到的“决策规则”（比如“看到红灯就刹车”“用户问事实就输出准确答案”）。

2、RL的核心特点（区别于其他机器学习）

1. **无监督标注**：不需要提前准备“输入 - 输出”的标注数据（比如监督学习需要的“图片 - 标签”），数据通过智能体与环境的交互实时生成；
2. **长期视角**：不追求单次动作的“即时奖励”，而是最大化“长期累积奖励”（比如自动驾驶不会为了短期加速而忽视长期安全）；

赞同 447

14 条评论

1095

20

分享

更多

关于作者



笨鸟先飞
持续学习~

车中草同学、郭达森、大模型面试官也关注了他

回答

7

文章

44

关注者

852

已关注

发私信

大家都在搜

换一换

美国总统特朗普结束访华 433 万

热

央视拿下世界杯版权 393 万

热

2人摆拍盲道被撞被采取刑... 388 万

新

马斯克：只有我和黄仁勋坐... 382 万

热

特朗普访华国际社会评价 340 万

热

雷军和马斯克合照 328 万

普京访华 327 万

中美元首举行会谈 321 万

河北无极县一火锅店发生刑... 316 万

热

OPPO 高级副总裁辞职直... 306 万

热



3、为什么需要RL

物理世界中，很多真实问题本质上就是「序列决策」，凡是符合以下几种情境的场景下，强化学习都天然适用：

- 机器人控制：机械臂抓取、无人机飞行、自动驾驶。
- 游戏 & 对弈：围棋、星际争霸、Dota2 (AlphaGo、AlphaStar)。
- 推荐与广告：不是只看「这一条推荐是否被点」，而是看 长期用户价值：留存、生命周期价值、多次交互。
- 运筹 & 调度：仓储选址、路径规划、资源调度（多少机器处理多少任务）。
- 对话系统 & Agent：一个 Agent 多轮对话、调用工具、写代码、检查结果，这些都是「长链路、多步反馈」的过程。

这些场景共性就是：

当下的选择会影响「未来能走到的状态」，而我们关心的是整体长期收益，不是某一步的得失。这类问题，用纯监督学习往往很难建一个特别合理的目标函数，RL 则是为这种情形量身定做的。

4、举例说明

下面使用一个悬崖漫步的例子说明一下强化学习。从4X12的网格左下角状态 (Initial State) 出发，目标是右下角的旗帜状态(Goal State)。智能体 (Agent) 可以采取4种动作(Action)：上、下、左、右，环境 (Environment) 中有一段是悬崖，智能体每走一步奖励 (Reward) 是-1，掉入悬崖是-100，掉入悬崖和到达终点都是终止态，会回到起点，而最终从起点到终点的最优路径就是策略 (Policy)。

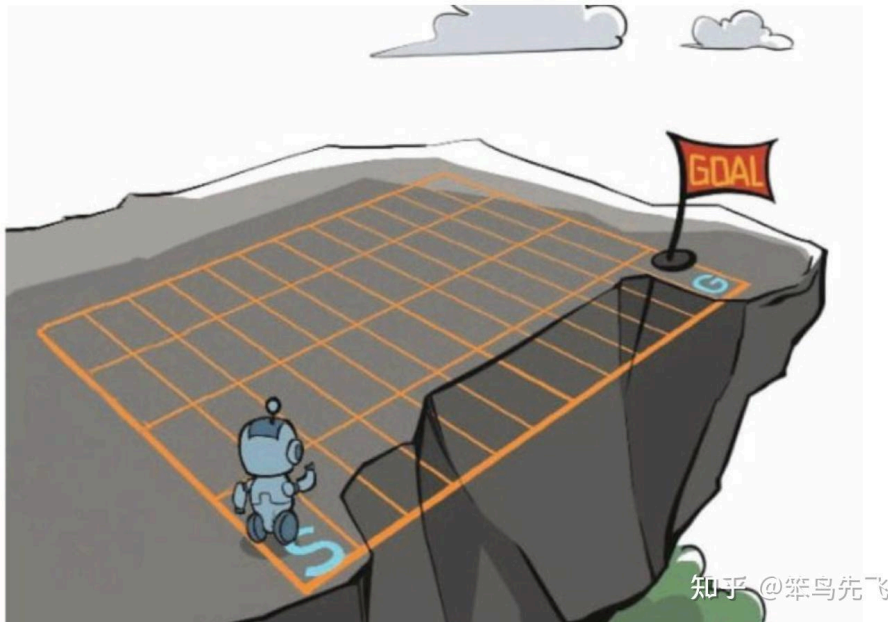


图2. RL示例

物理世界中，很多真实问题本质上就是「序列决策」，凡是符合以下几种情境的场景下，强化学习都天然适用：

除此之外，对大模型 / Agent 来说，RL更是「后训练」阶段的核心工具，其带来的好处包括：

1. 能直接优化「任务成功率」而不是「和标注相似度」。
2. 能允许模型在一些场景里探索新的策略，而不是拘泥于人类示范。
3. 天然适合「Agent + 工具 + 环境」的一整套闭环。

二、RL核心理论

1、问题建模：马尔可夫决策过程⁺ (MDP)

强化学习到底在学习什么？要想回答这个问题，我们可以将其抽象成一个经典MDP

(Markov Decision Process, 马尔可夫决策过程)，一个（折扣）马尔可夫决策过程通常写成一个5元组：

$$M=(S,A,P,R,\gamma)$$

其核心要素如下：

- 状态空间 S ：当前环境的刻画，比如棋盘布局、机器人位置、当前对话历史等。
- 动作空间 A ：在这个状态下，智能体能做的选择：走一步、说一句话、推荐一个商品、买/卖/不动...
- 转移概率 P ：给定当前状态和动作，下一状态的分布，其体现了系统的物理/业务演化规律，通常对智能体是未知的，表达形式如下：

$$P(s' \mid s, a) = \Pr(s_{t+1} = s' \mid s_t = s, a_t = a)$$

奖励函数 R ：环境给的一句「好/不好」的反馈，可以是立即的，也可以是很延迟的，常见写法如：

$$r(s, a) = \mathbb{E}[R \mid s_t = s, a_t = a]$$

折扣因子 $\gamma \in (0, 1)$ ：用来定义「未来奖励」的重要程度， γ 越接近 1 越重视长期收益， γ 越小越「短视」，只在乎眼前利益。

给定一个 MDP，要解决的核心问题是：**选什么动作？** 选取什么动作执行往往由策略 Policy $\pi(a \mid s)$ 决定，即给定状态下智能体选择动作的分布——这就是我们要学到的东西。

在策略 π 下，一次从开始到结束的交互形成一条轨迹 (trajectory)：

$$\tau=(s_0,a_0,r_1,s_1,a_1,r_2,s_2,...)$$

生成过程如下：

1. 初始： $s_0 \sim \rho_0$
2. 每一步：

- 策略选动作： $a_t \sim \pi(a_t \mid s_t)$
- 环境转移： $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$
- 环境给奖励： $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$

这条轨迹的概率：

$$p_{\pi}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

回报 Return G 定义为从当前时刻往后看的「总收益」，比如

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

其最终**目标**是：找到一个策略 π_{θ} ，最大化「期望回报」

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G]$$

为什么需要值函数：在别国的 MDP 里，我们的目标是最大化期望回报，但这个目标是「整条轨迹」级别的，不够“局部化”。为了能对“当前在某个状态/做某个动作”进行评估，我们引入**值函数 (value function)**：

值函数 = 在某个状态 (或状态 + 动作) 下，未来能拿到的“好处”的期望。

它把「整条未来」压缩成一个标量，方便比较、优化和做动态规划。

值函数有以下几种定义形式：

- **状态价值函数**：在状态 s 上，如果之后一直按策略 π 走下去，从现在开始往后能拿到的折扣总奖励的期望：

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[G_t \mid s_t = s \right]$$

状态-动作价值函数：在状态 s 下先执行一次动作 a ，然后以后都按策略 π 走下去，能拿到的折扣总奖励期望：

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[G_t \mid s_t = s, a_t = a \right]$$

优势函数：在状态 s 下，做动作 a 比起「随便按策略 π 正常走」到底好多少/差多少：

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

$A^{\pi}(s, a) > 0$ 说明比平均水平好， $A^{\pi}(s, a) < 0$ 说明比平均水平差，优势函数出现在很多策略梯度算法 (Actor-Critic⁺, PPO, A2C) 里，用来减少梯度估计的方差。

3、核心概念：Bellman期望方程

总体上，任何状态的回报都可以被拆解为两个部分：一是从当前状态到下一个状态的即时奖励；二是从下一个状态开始，按照特定策略行动，未来的折扣回报。值函数的关键性质是满足递归关系，而这种递归关系就是**Bellman 期望方程⁺** (Bellman Expectation Equation)。

- **对状态价值函数 $V^{\pi}(s)$ 的Bellman方程**

从定义出发：

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[G_t \mid s_t = s \right]$$

把 G_t 拆开：

$$G_t = r_{t+1} + \gamma G_{t+1}$$

代入：

$$\begin{aligned} V^{\pi}(s) &= \mathbb{E}_{\pi} \left[r_{t+1} + \gamma G_{t+1} \mid s_t = s \right] \\ &= \mathbb{E}_{\pi} \left[r_{t+1} \mid s_t = s \right] + \gamma \mathbb{E}_{\pi} \left[V^{\pi}(s_{t+1}) \mid s_t = s \right] \end{aligned}$$

再展开条件期望（先对动作再对下一个状态求期望）：

$$V^{\pi}(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left(r(s, a, s') + \gamma V^{\pi}(s') \right)$$

这就是 Bellman 期望方程 的离散形式。

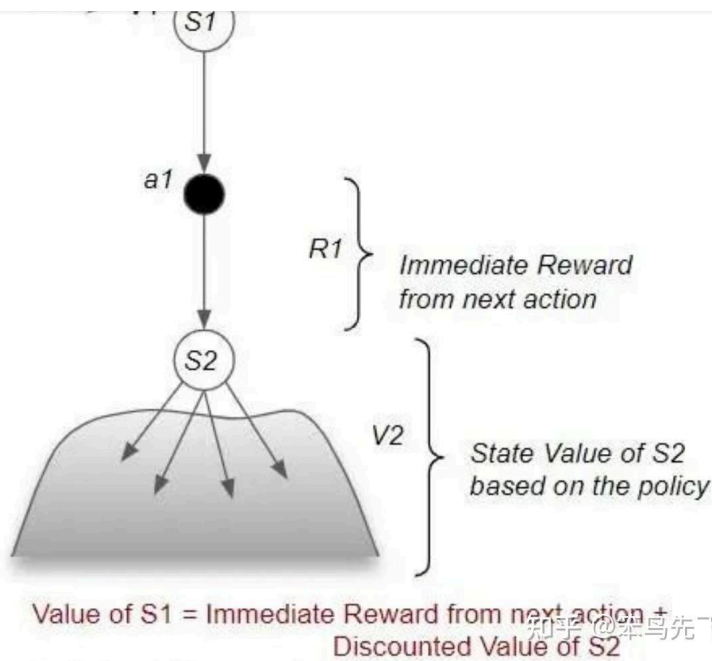


图3. 状态价值函数Bellman方程图例

• 对状态-动作价值函数 $Q^{\pi}(s,a)$ 的Bellman方程

类似地, 对 Q^{π} 有:

$$\begin{aligned} Q^{\pi}(s,a) &= \mathbb{E}_{\pi} \left[G_t \mid s_t=s, a_t=a \right] \\ &= \mathbb{E}_{\pi} \left[r_{t+1} + \gamma G_{t+1} \mid s_t=s, a_t=a \right] \\ &= \mathbb{E}_{\pi} \left[r(s,a,s') + \gamma \sum_{a'} P(a' \mid s,a) Q^{\pi}(s',a') \right] \end{aligned}$$

展开成求和形式:

$$Q^{\pi}(s,a) = \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma \sum_{a'} P(a' \mid s,a) Q^{\pi}(s',a') \right]$$

Bellman最优方程 (Bellman Optimality Equation)

上面是「给定策略」时的值函数方程, 如果我们关心的是最优策略 π^* , 则对应有**最优值函数**:

$$V^*(s) = \max_{\pi} V^{\pi}(s), \quad Q^*(s,a) = \max_{\pi} Q^{\pi}(s,a)$$

利用“最优策略在每一步都选那时最优动作”的直觉, 可以写出**Bellman最优方程**。

1. 对 $V^*(s)$ 的**最优方程**: 在状态 s 下, 选择那个能带来最高「立刻奖励 + 折扣未来价值」的动作:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma V^*(s') \right]$$

对 $V^*(s)$ 的**最优方程**:

$$Q^*(s,a) = \sum_{s'} P(s' \mid s,a) \left[r(s,a,s') + \gamma \max_{a'} Q^*(s',a') \right]$$

进一步可以得到**最优策略的显式形式**:

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

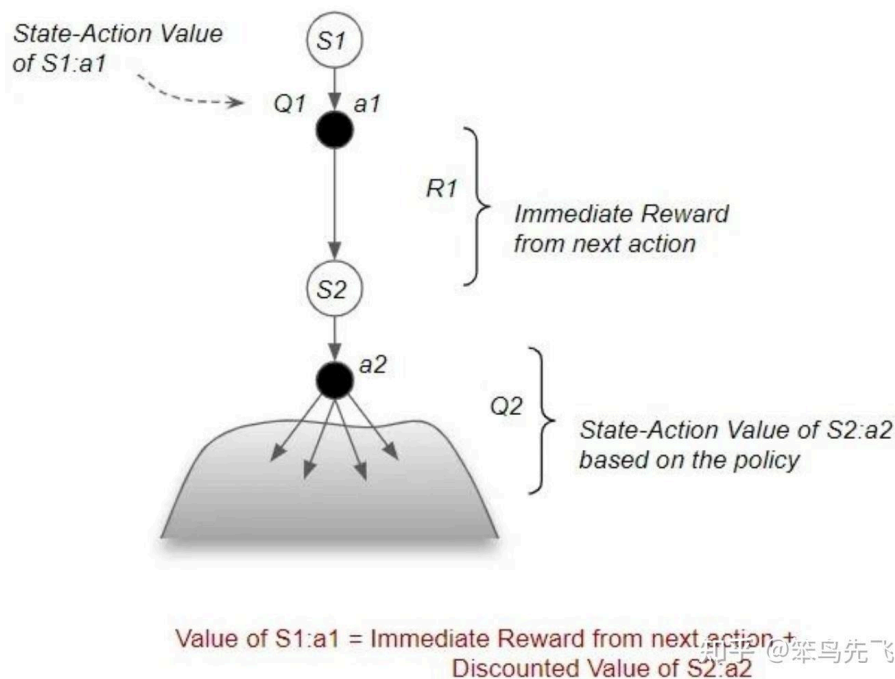


图4. 状态-动作价值Bellman方程图例

三、RL常用算法

1、常用算法分类

1. 从优化目标来看，常用RL算法包括以下几个类别：

- **基于价值函数的方法 (Value-Based)**：基于价值函数的方法，就是先学会“每个状态/动作有多好”（价值），再用这个价值函数去导出策略，其典型做法为学一个状态价值函数 $V(s)$ 或动作价值函数 $Q(s,a)$ ，而在深度 RL 时代，一般都学 Q 函数（因为更容易直接导出策略），常用算法有 Q-learning 等。
- **基于策略的方法 (Policy-Based)**：直接学一个 $\pi_{\theta}(s,a)$ ，把策略本身当成参数化模型，直接最大化期望回报 $J(\theta)$ ，常用算法有 REINFORCE⁺ 等。
- **策略价值并行方法 (Actor-Critic)**：同时学习策略 (Actor) 和价值函数 (Critic)，用价值函数做「baseline」减小方差，常用算法有 PPO 等。

1. 从数据来源来看，常用RL算法可分为以下两个类别：

- **On-Policy**：训练数据由需要训练的策略本身通过与环境的互动产生，用自己产生的数据来进行训练（可以理解为需要实时互动）。
- **Off-Policy**：同训练数据预先收集好（人工或者其它策略产生），策略直接通过这些数据进行学习。。

2、典型算法详解

本章节聚焦了一些深度RL领域的常见算法并介绍其理论依据，公式推导和代码实现。

2.1 Q-learning

Q-learning 的核心目标就是：在不知道环境转移概率 $P(s'|s,a)$ 的情况下，直接通过与环境交互采样到的 $(s_t, a_t, r_{t+1}, s_{t+1})$ ，用一种「自举 (bootstrapping)」的方式逼近 $Q^*(s,a)$ 。

Q-learning 的基本思想如下：

$$\text{目标值 } y_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a')$$

把这个 y_t 当作对 $Q^*(s_t, a_t)$ 的一个「软标签」，做一个逐步逼近：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \text{bigl}(y_t - Q(s_t, a_t) \bigr)$$

即：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right)$$

其中：

- α 是学习率；
- 括号里的量叫 TD 误差（Temporal-Difference error）：

$$\delta_t = r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)$$

Q-learning 每步都在用「目标 = 立即奖励 + 折扣后的下一状态最大 Q」来更新当前 Q。

假设：

状态空间 S 和动作空间 A 都是离散且可枚举；

用一个二维表 Qs 存储每个状态-动作对的 Q 值；

则 Q-learning 算法的伪代码如下：

输入：学习率 $\alpha \in (0, 1]$ ，折扣因子 $\gamma \in [0, 1]$ ，
探索系数 ϵ （可随时间衰减），
状态空间 S ，动作空间 A

初始化：对所有 $s \in S, a \in A$ ，令 $Q(s, a) \leftarrow$ 任意值（例如 0）

for episode = 1, 2, ... do
从环境中初始化状态 $s \leftarrow s_0$

while s 不是终止状态 do

1. 使用 ϵ -greedy 策略选动作
以概率 ϵ ：从 A 中随机选择动作 a
以概率 $1 - \epsilon$ ：令
 $a \leftarrow \operatorname{argmax}_{a'} Q(s, a')$

2. 与环境交互，获得下一步
执行动作 a ，观察到即时奖励 r 和下一个状态 s'

3. 计算 TD 目标和更新 Q
令
 $y \leftarrow r + \gamma \max_{a'} Q(s', a')$ （若 s' 为终止状态，则 $y \leftarrow r$ ）
更新：
 $Q(s, a) \leftarrow Q(s, a) + \alpha * (y - Q(s, a))$

4. 状态前移
 $s \leftarrow s'$

end while

end for

REINFORCE 算法核心在于「构建任何价值函数 / Q 函数，从而策略 $\pi_\theta(s)$ 构成一个马尔可夫决策模型，用「采样 + 回报」直接估计 $\nabla_\theta J(\theta)$ ，然后做梯度上升。

核心公式推导如下：

一条长度为 T 的轨迹：

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$$

在策略 π_θ 下的概率：

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t | s_t) P(s_{t+1} | s_t, a_t)$$

目标函数：

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta} [G_0] = \int p_\theta(\tau) G_0 d\tau$$

对 $J(\theta)$ 求梯度：

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) G_0 d\tau \\ &= \int \nabla_\theta p_\theta(\tau) G_0 d\tau \end{aligned}$$

利用 $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$ ：

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

接着展开 $\log p_\theta(\tau)$ ：

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t | s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t)$$

只有 $\log \pi_\theta$ 含 θ ，所以：

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t)$$

带回去：

$$\nabla_\theta J(\theta) = \mathbb{E} [G_0 \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t | s_t)]$$

通常我们把「从时刻 t 开始的未来回报」归到对应时刻 t 上，用 G_t 替代 G_0 得到更合理的 credit assignment：

$$\nabla_\theta J(\theta) = \mathbb{E} [\sum_{t=0}^{T-1} G_t \nabla_\theta \log \pi_\theta(a_t | s_t)]$$

这就是 **REINFORCE 的核心策略梯度公式**，即在某个状态下，如果这次行为后面的回报 G_t 很高，就增大它的 $\log$ 概率；反之就减小。

该算法伪代码实现如下：

算法 REINFORCE(α, γ)
初始化策略参数 θ （例如随机）

记录列表: $states = []$, $actions = []$, $rewards = []$

```
while episode 未结束:
    根据当前策略  $\pi_{\theta}(\cdot|s)$  采样动作  $a$ 
    执行动作  $a$ , 获得  $r, s'$ 
    将  $s, a, r$  追加到各自列表
     $s \leftarrow s'$ 

# 2. 计算每个时间步  $t$  的折扣回报  $G_t$ 
 $G \leftarrow 0$ 
returns = 空列表
对 rewards 从后往前遍历:
     $G \leftarrow r + \gamma * G$ 
    将  $G$  插入 returns 头部 # 得到  $[G_0, G_1, \dots, G_{T-1}]$ 

# 3. 计算梯度并更新  $\theta$ 
梯度估计  $g \leftarrow 0$ 
对每个时间步  $t$ :
     $g \leftarrow g + G_t * \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ 

 $\theta \leftarrow \theta + \alpha * g$ 
```

2.3 PPO

近些年来非常流行对大语言模型做“基于人类反馈”的强化学习微调（RLHF），其核心流程是：先有一个预训练语言模型（或初步监督微调好的 SFT 模型），再结合人类偏好或自动奖励模型，对其进行策略优化，PPO就是该系列的主力算法。PPO（Proximal Policy Optimization）是 OpenAI 在 2017 年提出的一种策略优化（Actor-Critic）算法，专注于简化训练过程，克服传统策略梯度方法（如TRPO）的计算复杂性，同时保证训练效果。

- 问题：在强化学习中，直接优化策略会导致不稳定的训练，模型可能因为过大的参数更新而崩溃。
- 解决方案：PPO通过限制策略更新幅度，使得每一步训练都不会偏离当前策略太多，同时高效利用采样数据。

假设你是一个篮球教练，训练球员投篮：

- 如果每次训练完全改变投篮动作，球员可能会表现失常（类似于策略更新过度）。
- 如果每次训练动作变化太小，可能很难进步（类似于更新不足）。
- PPO 的剪辑机制就像一个“适度改进”的规则，告诉球员在合理范围内调整投篮动作，同时评估每次投篮的表现是否优于平均水平。

PPO遵从On-Policy的策略，On-Policy的策略一般由四个关键组件组成训练的pipeline：

- Actor: 产生动作的策略，最终需要学习得到的model。
- Critic: 评估动作或状态的价值网络，预测生成一个token后，后续能带来的收益。
- Reward Model: 对状态转移给出即时的奖励的模型或者函数，输入query 和response，输出一个得分。
- Reference Model: 参考模型，通常是sft 后的model，这是为了防止在训练过程中，策略网络在不断的更新后，相对于原始策略偏移地太远（避免它训歪了）。

PPO RLHF pipeline 可分为以下三步走：

1. SFT: 用人类写的示例做监督微调得到 π_{sft} ；
2. RM: 基于成对偏好 $(x, y^+ + y^-)$ 训练一个 reward model $r_{\phi}(x, y)$ ；
3. RL: 优化策略 π_{θ} ，优化目标如下：

PPO 的完整损失函数使用的是一个联合损失：

$$L(\theta) = \mathbb{E}_t \left[L_t^{\text{CLIP}}(\theta) - c_1 \cdot L_t^{\text{VF}}(\theta) + c_2 \cdot S[\pi_\theta](s_t) \right]$$

实现时一般是最大化上述公式，或者最小化其负数。

损失函数可拆解为以下三项 Actor + Critic + Entropy：

1. 策略损失 (Actor)

$$L_t^{\text{CLIP}}(\theta) = \min \left(r_t(\theta), \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)} \right) \cdot \hat{A}_t$$

这是PPO 提出的核心目标函数 clipped surrogate objective，其目的在于用 clip 替代显式 KL 约束；

其中：

· $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\text{old}}(a_t | s_t)}$ 为「重要性采样」比率；

· ϵ 是一个小常数（如 0.1 或 0.2），控制「信任域」宽度；

· $\text{clip}(x, 1-\epsilon, 1+\epsilon)$ = 把 x 限制在 $[1-\epsilon, 1+\epsilon]$ 区间内；

直觉上可分为以下两种情况来看：

1. 当 $\hat{A}_t > 0$ （这次动作比平均好），我们希望 $r_t > 1$ 来提高该动作概率，但如果 $r_t > 1 + \epsilon$ ，就认为「涨太狠了」，用 $1 + \epsilon$ 截断；
2. 当 $\hat{A}_t < 0$ （这次动作比较差），我们希望 $\hat{A}_t < 0$ 来降低该动作概率，但如果 $\hat{A}_t < 0$ ，就认为「减太狠了」，用 $1 - \epsilon$ 截断；

核心思想为在比率偏离旧策略太远时，进一步优化会被截断，损失不再鼓励大步更新。

1. 价值函数损失 (Critic)

$$L_t^{\text{VF}}(\theta) = \left(V_\theta(s_t) - \hat{G}_t \right)^2$$

PPO 不是只优化策略，还会同时学习上述 value function；

1. 熵奖励 (Entropy Bonus)：

$$S[\pi_\theta](s_t) = \text{Entropy}(\pi_\theta(\cdot | s_t))$$

PPO 不是只优化策略，还会同时学习上述 value function；

实际代码里，一般写成最小化如下 loss：

$$\text{loss} = -\mathbb{E}_t \left[L_t^{\text{CLIP}}(\theta) \right]$$

- $c_1 \cdot \mathbb{E}_t \left[L_t^{\text{VF}}(\theta) \right]$
- $c_2 \cdot \mathbb{E}_t \left[S[\pi_\theta](s_t) \right]$

PPO 算法伪代码实现如下：

```
loop: # 每一轮迭代
# ===== 1. Rollout 收集数据 =====
trajectories = []
for env_step in range(T): # 也可以多环境并行
    s_t = 当前状态
    a_t ~  $\pi_{\theta}(\cdot | s_t)$ 
    执行 a_t 得到 (r_{t+1}, s_{t+1}, done)
    记录 (s_t, a_t, r_{t+1}, done, log  $\pi_{\theta}(a_t | s_t)$ , V_φ(s_t))

    if done: 重置环境

# ===== 2. 计算优势和回报 =====
用 GAE(γ, λ) 从后往前计算 A_t
用 G_t = A_t + V_φ(s_t) 作为回报目标
对 A_t 做归一化

# ===== 3. 多 epoch, 小批次优化 =====
for k in range(K): # K 个 epoch
    对 trajectories 打乱并按 batch_size 分组
    for 一个 minibatch B:
        从 B 中取出 s, a, A, G, logπ_old, V_old

        # 重新算当前策略的 log prob
        logπ_new = log  $\pi_{\theta}(a | s)$ 
        r = exp(logπ_new - logπ_old)

        L_clip = mean( min( r * A, clip(r, 1-ε, 1+ε) * A ) )

        V_new = V_φ(s)
        value_loss = mean( (V_new - G)^2 )

        entropy = mean(策略熵)

        loss = -L_clip + c1 * value_loss - c2 * entropy

    对 (θ, φ) 进行一次梯度下降
```

2.4 DPO

鉴于经典 RLHF pipeline (以 PPO 为例), 在 RL 优化策略的步骤同时要在线采样、计算 value function、advantage、clip...工程上比较重。DPO (Direct Preference Optimization) 算法提出其核心主张: KL 正则的 RLHF 目标, 其实可以在闭式下解出「最优策略的形式」, 然后直接用分类损失 / logistic loss 去拟合这个最优策略, 完全不用显式 reward model, 也不用 RL 采样。

DPO 与 RLHF 算法对比如下:

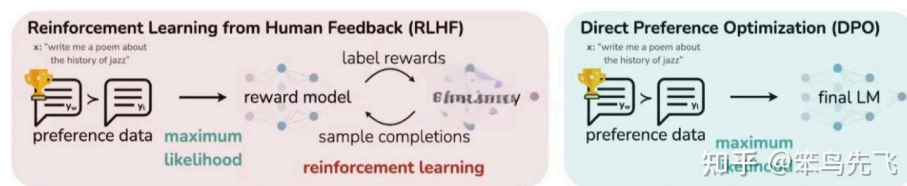


图5. DPO 与 RLHF 算法对比

DPO 的核心特点如下:

- 仍然用偏好数据 (x, y^+, y^-) ;
- 不训练单独 reward model, 不跑 PPO/TRPO;
- 只需要两个 LM: 参考模型 π_{ref} (通常是 SFT) + 可训练模型 π

与模型成本成正比。 — Online preference based 万夫喊狗。

DPO 算法核心公式推导如下：

1. 首先从标准 KL 正则 RLHF 目标出发，我们希望找一个策略 $\pi_{\theta}(y|x)$ ，最大化

$$\mathbb{E}_{y \sim \pi(\cdot|x)} [r(x,y) - \beta^{-1} \mathbb{E}_{y \sim \pi_{\text{ref}}(\cdot|x)} [r(x,y)]]$$

展开成显式求和

$$\max_{\pi} \sum_y \pi(y) [r(y) - \beta^{-1} \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\text{ref}}(y)}] \quad \text{s.t.} \quad \sum_y \pi(y) = 1.$$

这里 π_{ref} 是参考策略（一般是 SFT 模型）， $\beta > 0$ 用于控制 KL 正则强度。

1. 对 π 做变分求解 → 最优策略形式

对每个 y 令偏导为 0，可以得到：

$$\log \pi^*(y|x) = \log \pi_{\text{ref}}(y|x) + \beta r(x,y) + C(x)$$

即：

$$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp(\beta r(x,y))$$

从上式反推 reward 与 log 概率差 的关系：

$$\beta r(x,y) = \log \pi^*(y|x) - \log \pi_{\text{ref}}(y|x) + C(x) \quad (*)$$

偏好数据 + Bradley-Terry 模型构建 DPO 损失

标注数据是成对偏好 (x, y^+, y^-) ，假设

$$\Pr(y^+ \succ y^- | x) = \frac{\exp(\beta(r(x, y^+) - r(x, y^-)))}{\exp(\beta(r(x, y^+) - r(x, y^-))) + \exp(\beta(r(x, y^-) - r(x, y^+)))}$$

用 (*) 写成「log 概率差」形式（常数抵消）：

$$\beta(r(x, y^+) - r(x, y^-)) = \log \frac{\pi^*(y^+|x)}{\pi^*(y^-|x)} - \log \frac{\pi_{\text{ref}}(y^+|x)}{\pi_{\text{ref}}(y^-|x)}$$

用当前模型 π_{θ} 逼近 π^* 求解 DPO 损失

用 π_{θ} 逼近 π^* ，得到偏好概率近似：

$$\Pr_{\theta}(y^+ \succ y^- | x) \approx \frac{\pi_{\theta}(y^+|x)}{\pi_{\theta}(y^+|x) + \pi_{\theta}(y^-|x)} \approx \frac{\pi_{\theta}(y^+|x)}{\pi_{\theta}(y^+|x) + \pi_{\theta}(y^-|x)}$$

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y^+, y^-)} [\log \sigma(\beta(\log \frac{\pi_{\theta}(y^+|x)}{\pi_{\theta}(y^-|x)} - \log \frac{\pi_{\text{ref}}(y^+|x)}{\pi_{\text{ref}}(y^-|x)}) + \log \frac{\pi_{\text{ref}}(y^+|x)}{\pi_{\text{ref}}(y^-|x)})]$$

DPO 算法伪代码实现如下：

输入：

- 参考模型 π_{ref} （冻结参数）
- 可训练模型 π_{θ} （初始参数 = π_{ref} ）
- 偏好数据集 $D = \{(x, y_{\text{pos}}, y_{\text{neg}})\}$
- 超参数: β , learning_rate , batch_size , num_epochs

```

for epoch in 1..num_epochs:
    对 D 打乱并按 batch_size 划分
    for (x_batch, y_pos_batch, y_neg_batch) in mini-batches:

        # ----- 1. 计算 log prob -----
        # 对正样本
        logp_pos_theta = log  $\pi_{\theta}(y_{\text{pos}} | x)$       # shape: [B]
        logp_pos_ref   = log  $\pi_{\text{ref}}(y_{\text{pos}} | x)$    # shape: [B]

        # 对负样本
        logp_neg_theta = log  $\pi_{\theta}(y_{\text{neg}} | x)$       # shape: [B]
        logp_neg_ref   = log  $\pi_{\text{ref}}(y_{\text{neg}} | x)$    # shape: [B]

        # ----- 2. 构造  $\Delta \log \text{prob}$  -----
        delta_theta = logp_pos_theta - logp_neg_theta    #  $\Delta \log \pi_{\theta}$ 
        delta_ref   = logp_pos_ref   - logp_neg_ref     #  $\Delta \log \pi_{\text{ref}}$ 

        # ----- 3. DPO logistic loss -----
        logits =  $\beta * (\text{delta\_theta} - \text{delta\_ref})$   # shape: [B]

        # 概率目标:  $P(y_{\text{pos}} \text{ 被选中 }) = \sigma(\text{logits})$ 
        # 负对数似然:
        dpo_loss = - mean( log  $\sigma(\text{logits})$  )

        # （有些实现会再加一个对称项  $-\log(1-\sigma(\text{logits}))$ ，本质等价）

        # ----- 4. 反向传播 & 更新 -----
        loss = dpo_loss
        对  $\theta$  做一次梯度下降更新
  
```

2.5 GRPO

GRPO (Group Relative Policy Optimization) 是DeepSeek提出的强化学习算法，专为优化大语言模型（如DeepSeek-V3）设计。它通过组内相对奖励代替传统价值模型，降低训练成本，同时保持策略稳定性。

GRPO 与 PPO 算法的流程对比如下：

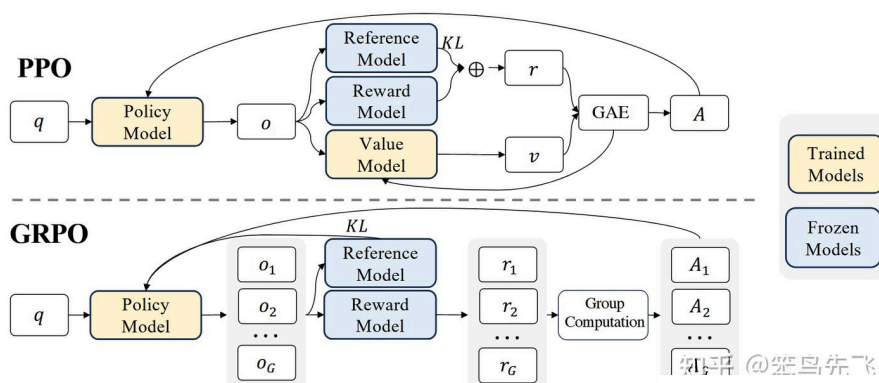


图6. GRPO VS PPO

不再用 critic, 不再加 value function,

- 每个 prompt 一次采样一组输出 (group), 用组内的平均 reward 当 baseline;
- 优势 A 完全由「相对于组平均的 reward」来计算;
- 仍然保留 PPO 的 clip 比例 和 KL 正则, 更新稳定性不丢。

GRPO 的最终目标: group + 无 critic + KL 直接进 loss

相较于 PPO, GRPO 做了三件关键改动:

1. 每个问题 q 不再只采一个输出, 而是从旧策略 $\pi_{\theta_{\text{old}}}$ 采样一个组 $\{o_1, \dots, o_G\} \sim \pi_{\theta_{\text{old}}}(\cdot \mid q)$ 。
2. 用 reward model 对这 G 个输出打分, 算出组内的相对奖励 $\rightarrow$ 得到优势 $\pi_{\theta_{\text{old}}}$, 完全基于组内比较 (下面详细展开)。
3. KL 惩罚不再塞进 reward, 而是作为一个显式的正则项 $-\beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ 。

GRPO 优化主目标如下:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}_{i=1}^G} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left(\min \left(r_{i,t}(\theta), 1 + \epsilon \right), \operatorname{clip} \left(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon \right) \right) \hat{A}_{i,t} \right] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

其中

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}$$

可以看到: 结构上就是 PPO + KL, 但优势 $\hat{A}_{i,t}$ 完全来自 group-relative reward, 不再用 value network。

KL 项使用一个无偏估计:

$$\widehat{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_{\theta}(o_{i,t})} \log \frac{\pi_{\text{ref}}(o_{i,t})}{\pi_{\theta}(o_{i,t})}$$

保证非负。

组相对优势 $\hat{A}_{i,t}$ 怎么算

GRPO 的“精髓”就在 $\hat{A}_{i,t}$ 上: 让 group 自己提供 baseline, 代替 value function。

对每个问题 q , 有一组输出 $\{o_1, \dots, o_G\}$;

相对地有 reward model 打分 $\{r_1, \dots, r_G\}$;

使用过程监督对这组 reward 做 group 标准化 (目的是为了对数学 / 推理任务进行精细的监督, 每一步推理都给 reward)

- 把一个回答 o_i 划成多步 reasoning, 「每一步终止 token」记为 $\text{index}(j)$;
- 为每步打分 $r_i^{(j)}$ 得到一堆 step-level reward, 之后做 group 标准化;

$$\tilde{r}_i^{(j)} = \frac{r_i^{(j)} - \operatorname{mean}(R)}{\operatorname{std}(R)}$$

这里 R 是全组所有 step reward 的集合;

对任意一个 token t , 它所属输出 i 的优势是该 token 之后所有 step 的标准化 reward 之和:

给定:

- 初始策略模型 π_{θ_init} (通常是 SFT checkpoint)
- 奖励模型 r_{ϕ}
- 参考模型 π_{ref} (初始 = π_{θ_init})
- 任务 prompt 集合 D
- 超参数: ϵ (clip), β (KL), G (group size), μ (每批上内循环步数)

初始化 $\theta \leftarrow \theta_{init}$

```
for outer_iter = 1..I:
    设置参考模型  $\pi_{ref} \leftarrow \pi_{\theta}$       # 冻结一份
    for step = 1..M:
        从  $D$  采一批 prompt:  $\{q\}$ 
        设  $\pi_{old} \leftarrow \pi_{\theta}$       # 用当前策略作 roll-out 策略

        # ----- 1. 采样 group 输出 -----
        对每个  $q$ :
            采样  $G$  个输出  $\{o_i\} \sim \pi_{old}(\cdot | q)$ 

        # ----- 2. 计算 group reward -----
        用  $r_{\phi}$  对所有  $(q, o_i)$  打分, 得到  $\{r_i\}$  或 step-level reward

        # ----- 3. 计算组相对优势  $\hat{A}_{\{i,t\}}$  -----
        - outcome RL:  $\hat{A}_{\{i,t\}} = (r_i - \text{mean}(r)) / \text{std}(r)$ 
        - process RL: 用所有 step reward 标准化后, 令
             $\hat{A}_{\{i,t\}} = \text{sum}_{\{\text{future steps}\}} \text{normalized\_reward}$ 

        # ----- 4. policy update: 多次 GRPO 内循环 -----
        for k = 1.. $\mu$ :
            对这一批  $(q, \{o_i\})$  计算:
                - 比率  $r_{\{i,t\}} = \pi_{\theta} / \pi_{old}$ 
                - PPO-style clip surrogate using  $\hat{A}_{\{i,t\}}$ 
                -  $KL(\pi_{\theta} || \pi_{ref})$ 
            形成  $J_{GRPO}(\theta)$ , 对  $-J_{GRPO}(\theta)$  做一次梯度下降
```

三、LLM-RL VS Agentic-RL

1、LLM-RL (目前主流的 RLHF / PPO 微调)

LLM-RL 典型形态如下:

- 模型: 一个大语言模型 π_{θ} , 输入 prompt, 输出一整段回答;
- 环境: 几乎没有显式环境, 更多是“离线日志 + 打分器 (RM)”模式;
- 奖励: 人类偏好/排名 (RM 输出的标量) 或者简单功能性 reward (例如 code 能运行、数学题对不对);
- 算法: PPO / DPO / RPO / GRPO 一类的「对整段回答的概率分布做调整」。

可以粗暴理解为: 把 LM 当成一个大 policy, 每次行动就是“生成一整个回答”, 然后根据这次回答的评分, 整体推一下参数。

基本特征如下:

- 单轮或短上下文;
- 没有显式状态转移 (环境不会因为你这次回答改变「可观测状态」);
- 没有真正意义上的探索策略, 只是从现在的 LM 采样几条候选。

token 的结束。环境具头就是 Reward Model + Reference Model，这是一个虚拟的、静心的数学环境，优化的是文本的概率分布。

简单举个例子，LLM-RL 架构就是一个“带私教的模拟考试”系统：

1. **学生 (Actor)**：也就是我们要训练的 LLM，负责答题。
2. **考官 (Reward Model)**：代表人类喜好，只在最后**打一个总分**（比如：这篇 80 分）。
3. **紧箍咒 (Ref Model)**：防止学生为了刷分而走火入魔（乱凑字数），强迫它**保持正常说话**的习惯。
4. **私教 (Critic)**：因为考官只给总分，私教负责**实时预测分数**，一步步告诉学生：“刚才那句写得**好**，继续保持；这句写得**烂**，下次改掉”。

一句话总结：学生 (Actor) 在 私教 (Critic) 的指点下，努力讨好 考官 (Reward) 拿高分，同时还得戴着 紧箍咒 (Ref) 别乱写。

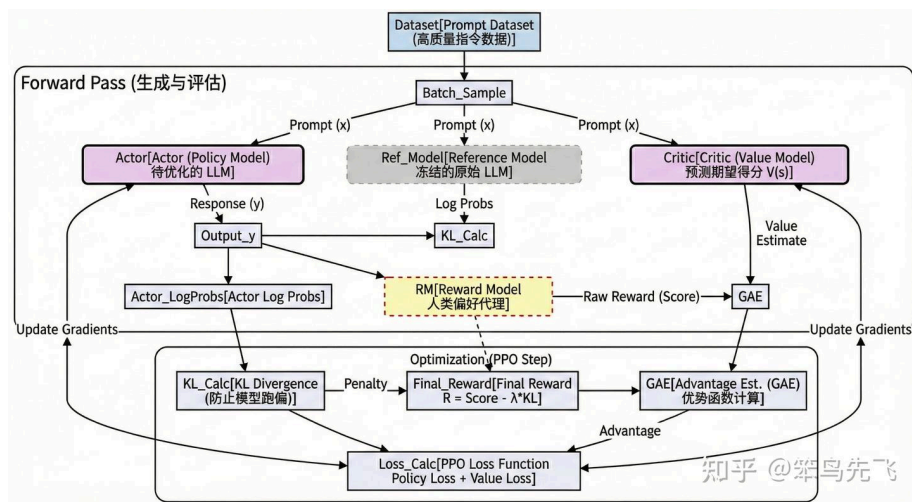


图7. LLM-RL流程图

2、Agentic-RL (基于智能体的强化学习)

这里的「Agent」指的是：

- 状态 s_t ：包含环境信息 + Agent 内部记忆 (history、工具输出、数据库状态...)；
- 动作 a_t ：不再只是“下一个 token”，而是：选择工具、构造 SQL / API 调用、规划子任务、决定是否继续对话、是否写入知识库等等；
- 环境 E ：真实的数据库、Web API、用户、任务队列、文件系统.....会随着动作变化；
- 回报 r_t ：和任务成功率、延迟、成本、用户满意度、安全约束相关；
- 策略：可以由 LLM+工具组成，但 RL 优化的是「整个决策流程」。

一句话总结，Agentic RL = 在“状态-动作-环境反馈”这个闭环上做 RL，LLM 只是这个闭环里实现策略的一部分。

这时候 LLM 不再仅仅是“嘴巴”（生成文本），而是成了“大脑”（决策中心），它通过操纵“四肢”（工具/API）与“世界”（环境）交互，并根据“绩效指标”（Reward）来优化自身的决策逻辑，如下图所示。

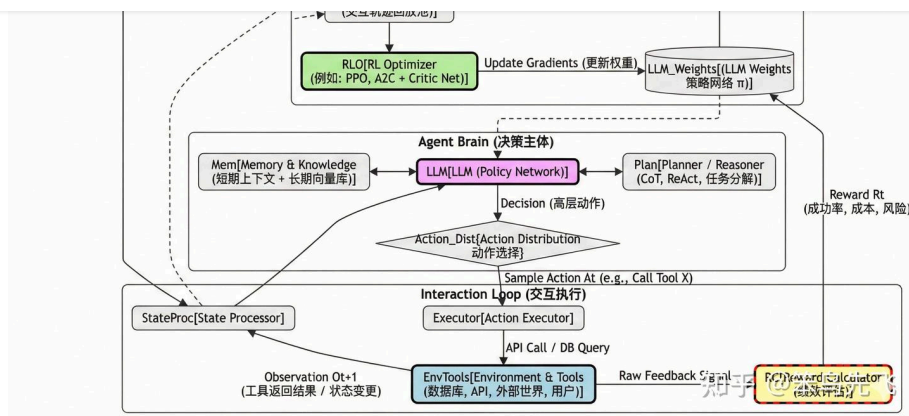


图8. Agentic-RL流程图

3、LLM-RL vs Agentic-RL 关键差异

3.1 环境 & 交互形式

LLM-RL:

- 环境基本是静止的：给你一个 prompt，你吐一个回答，结束；
- reward 在 “episode 终点” 给（整条回答一个分）；
- 不存在 “对同一个任务多轮试错” 这个概念。

Agentic-RL:

- 环境是动态的：查询数据库会改变上下文；调用 API 可能改变外部世界；用户下一句话取决于你刚刚的回答；
- 回合可以很长，多步骤、多工具、多轮对话；
- 需要通过多轮 trial-and-error 去发现更好的策略；

换句更尖锐的话：LLM RL 优化的是「一次性吐答案」的质量；而Agentic RL 优化的是「多步交互过程」本身。

3.2 行动粒度 & 信用分配 (credit assignment)

LLM-RL:

- 行动粒度 = token 或整段回答；
- reward 通常只在「最后」给一次（正确/错误、人类偏好分）；
- 信用分配基本是「把奖励摊到所有 token 上」，最多用 GAE 平滑一下；

Agentic-RL:

- 行动是高层决策：调用哪个 tool、读哪张表、如何规划子问题、是否结束任务；
- reward 可以在流程中的多个关键节点给（找到正确子问题、选中对的表 / API、成功更新知识库等等）；
- 信用分配可以精准到「哪一步决策让任务走向成功/失败」；

对「数据 Agent / 工具 Agent」来说：真正重要的是 “每一步选的工具和操作是否对任务有贡献”，这个粒度上，单纯对最终回答打个分再 PPO 一下，是很难学到东西的。

3.3 优化目标：输出分布 vs 任务绩效

LLM-RL:

- 目标多是「对齐」，而且在 给定 prompt、一次回答 这个框架里；
- reward 模型学的是「用户更喜欢哪种回答」；

- 目标更广泛「系统级 RL」，目标成功干（任务完成 / 回答干 / 正确干 / 成本（调用工具次数、API 费用、延迟）、稳定性 & 安全性（不会乱改数据、不会泄露隐私）；
- 甚至是多目标加权： $R = \alpha \cdot \text{成功率} - \beta \cdot \text{成本} - \gamma \cdot \text{风险}$

也就是说，LLM-RL 优化的是「回答好不好」；而 Agentic RL 优化的是「整个系统做事情做得好不好」。

3.4 数据来源 & 学习范式

LLM-RL:

- 典型 RLHF 是「离线数据 + 少量在线采样」；
- 主数据是标注好的对话 / 偏好对，环境不会变；
- 很多时候更像「加了 KL 正则的监督学习」→ DPO、IPO 等。

Agentic RL:

- 必须和环境「长期在线交互」才能形成 data flywheel，包括收集成功/失败信号、用户显式/隐式反馈，以及 on-policy 或 off-policy 地持续更新策略；
- 会涉及：探索、分布偏移、off-policy 修正等更“正统”的 RL 问题。

四、为什么 Agentic RL 是“必要的”

1、真实业务任务大多是“长过程 + 多工具”的

目前广泛关注的 **数据 Agent**，本质就是：给模型一个复杂任务（报表、诊断、数据质检...），它需要自己规划步骤、查表、连接 DB、抽取字段、写回结果，有时还要问人、有时要回滚操作。

而这些任务的成功与否：

- 完全取决于一连串决策的组合（选对/错工具、读对/错表、update 对/错字段.....）；
- 单次自然语言回答的 reward，只能部分反映这些决策好坏。

如果只在“最后一句回复”上做 PPO/RLHF:

- 模型学会的是「如何描述自己正在做什么」，
- 不一定学会「真正正确地做什么」。

这也就是为什么在 agent 场景里容易出现：嘴上说得天花乱坠，实际上工具调用乱来。

2、靠静态偏好数据，无法逼出“结构化策略”

很多 Agent 能力是「结构」而不是「风格」：

- 如何把一个复杂 query 分解成子任务；
- 如何根据 schema 选择合适的表；
- 如何在工具失败时重试 / fallback；
- 如何在 budget 约束下做最优查询计划。

这些东西：

- 一方面很难事先写成「成对偏好数据」；
- 另一方面更难靠标注几条“正确轨迹示例”去做 SFT 就学会泛化；

Agentic-RL 的必要点在于：

5. 这样策略「云自动发现」哪些规划/工具使用模式更成功」；

3、数据飞轮 & 在线学习：只有 Agentic-RL 能真正闭环

想象你有大量真实用户在用你的数据 Agent：

- 每天海量的对话 + 工具调用日志；
- 每条任务最终要么成功要么失败，并带有一些可观测 signal（用户是否继续追问、是否导出报表、是否投诉...）；

如果只做 LLM RL：

- 这些 log 大多被当成“提示工程素材”或者“再标注一点偏好对”；
- 更新节奏很慢，反馈利用率极低；

如果做 Agentic RL：

- 可以把这些日志直接变成 RL episode：其中 s_t 为 prompt + 历史交互 + 工具结果， a_t 为当前工具/操作/回复； r_t 为即时/终局任务得分；
- 用 off-policy AC / Q-learning / policy gradient 等方法持续更新；
- 形成真正的「Online Learning / 数据飞轮」；

在竞争场景下，这个“自动变聪明”的闭环是决定性差异，单纯 LLM-RL 做不到。

总结一下，传统的 LLM RL（例如 PPO-based RLHF）本质上仍然是一种“分布对齐”技术：它在离线偏好数据和静态 prompt 环境中，调整语言模型的输出概率分布，使单轮回答更符合人类偏好。然而，在现实应用中，真正具有商业价值的智能系统往往是 Agent 化的：它们需要在一个动态环境中进行多步决策、调用多种工具、维护长期记忆，并对任务成功率、成本、安全约束等系统级指标负责。这种情况下，仅仅针对单轮输出做 LLM RL 已经不够，我们需要将 RL 扩展到整个“状态-动作-环境反馈”的闭环上，用 Agentic RL 直接优化智能体的行为策略。换言之，LLM RL 让模型“说得更好”，而 Agentic RL 让系统“做得更好”；只有两者结合，才能支撑未来复杂的数据智能体和企业级 Agent 应用。

五、热门 Agentic RL 训练框架

1、Hugging Face TRL：LLM 强化学习的工业标准

Github: [github.com/huggingface/...](https://github.com/huggingface/trl)

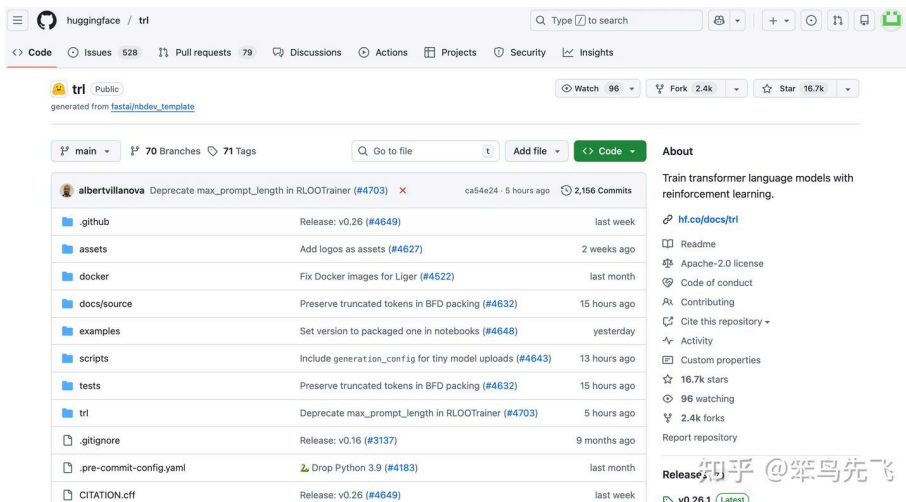


图9. Hugging Face TRL库

括 OpenAI DPO、Anthropic HH 模型）都可任 RL 上复现。其设计目标是将强化学习与 Transformers 生态无缝结合，为模型提供从 SFT → Reward Model → PPO/DPO 优化的全流程工具链。

核心功能

1. 算法支持：完整支持PPO、DPO、KTO、RLOO等主流强化学习优化算法。
2. 模型适配：可基于任何因果语言模型进行训练，并提供 AutoModelForCausalLMWithValueHead 模块，用于附加价值头以估计生成序列的回报。
3. 流程整合：内置标准RLHF示例管线，能够一体化执行从奖励模型训练到策略微调的全过程。

技术特点与用途

1. 研究基线：被广泛视为RLHF研究的基线框架，OpenAI、Anthropic等相关研究均可在其上复现。
2. 模块化设计：支持高度灵活的模块化实验配置，便于自由替换奖励模型、参考模型及优化算法组件。
3. 稳定与生态：训练稳定性高，且依托活跃的Hugging Face社区，非常适合需要可靠实现的前沿科研与企业级LLM对齐任务。

2、ms-swift: ModelScope开源的轻量化微训练部署框架

Github: [github.com/modelscope/m...](https://github.com/modelscope/ms-swift)

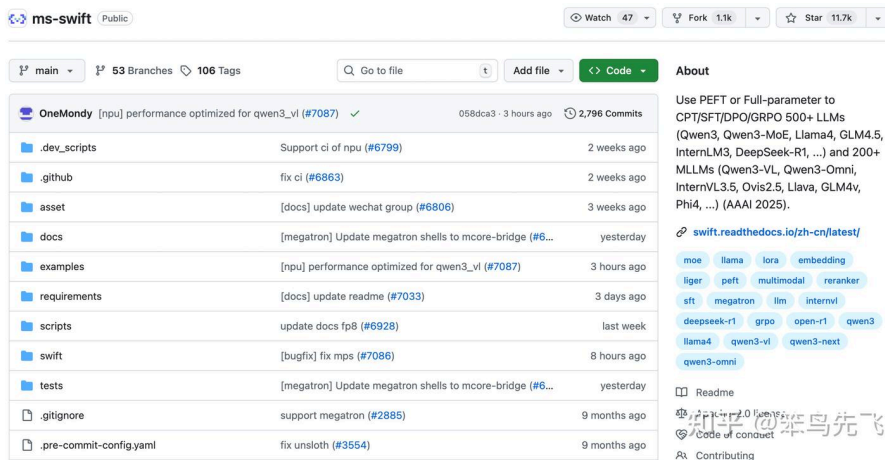


图10. ms-swift库

定位：ms-swift (Scalable lightWeight Infrastructure for Fine-Tuning) 是ModelScope（魔搭）社区开源的一个全链路、轻量化、可扩展的大模型训练部署框架。目标是通过参数高效训练技术，让研究者和开发者在有限资源下（如单张消费级显卡）也能高效地进行模型定制、对齐和部署。

核心功能

1. 丰富训练范式：支持全参数、轻量微调、人类对齐及强化学习训练。
2. 广泛模型与数据集支持：支持超600个纯文本和300个多模态模型，内置超150个数据集。
3. 全流程工具链：覆盖训练、推理、评测、量化、部署全流程，并提供Web-UI界面。
4. 多硬件与分布式支持：支持从消费级显卡到A100/H100及国产硬件的广泛环境，集成多种并行训练技术。

技术特点与用途

- 2. 专项优化技巧：采用多头例升打木件、开刀木件、多靶更新等技巧，显著提升强化学习训练速度。
- 3. 支持多模态与Agent训练：支持图文、视频等多模态模型的强化学习训练，并提供Agent训练模板，方便构建工具调用能力。

3、verl：字节跳动的生产级强化学习框架

Github: github.com/volcengine/v...

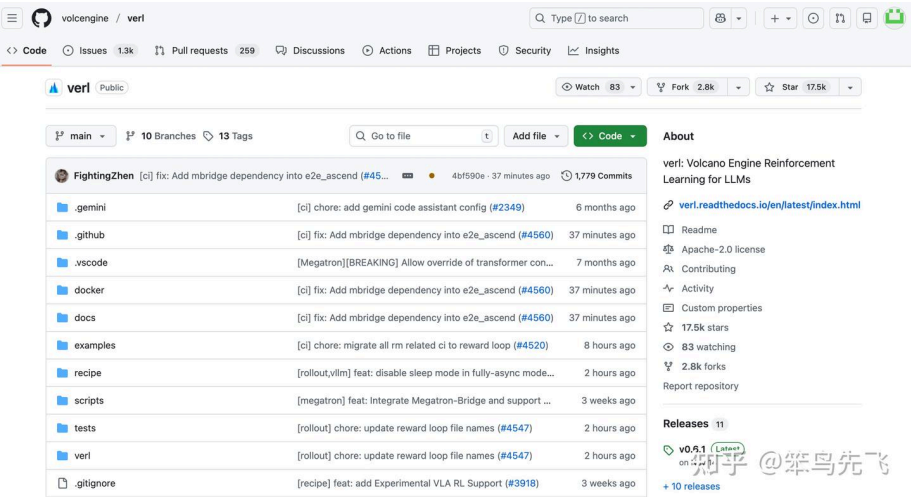


图11. verl库

定位： veRL (Volcano Engine Reinforcement Learning) 是字节跳动火山引擎于 2024 年底开源的分布式大模型强化学习训练框架。其设计目标是将 RLHF 的科研实现转化为可规模化部署的生产级系统。

核心功能

- 1. 提供覆盖完整训练流程的核心模块，包括Rollout生成器、奖励建模器、策略更新器和分布式调度器。
- 2. 支持PPO、DPO以及其自研的DAPO（动态对齐策略优化）和GRPO等多种算法，并通过异步流水线架构提升整体训练效率。
- 3. 其设计借鉴了工业级RL系统的思想，具备在数百张GPU集群上进行分布式训练的能力。

技术特点与用途

- 1. 生产级分布式训练：专为企业和研究机构的大规模模型后训练场景设计，采用任务并行、异步更新与奖励缓存等机制，显著降低GPU闲置率，优化资源利用率。
- 2. 高效的算法与应用：其内置的DAPO算法已被广泛应用于Qwen等主流模型系列中，有效优化了模型的推理稳定性和语言一致性，体现了其生产落地价值。

4、ART (Agent Reinforcement Trainer)：智能体行为优化框架

Github: github.com/OpenPipe/ART

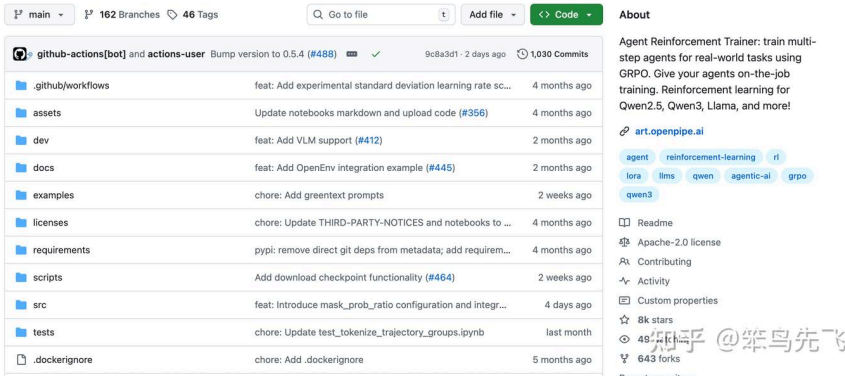


图12. ART库

定位：ART 是由 OpenPipe 团队在 2025 年推出的开源框架，专门面向 Agent 场景下的强化学习训练。它让语言模型在动态环境中执行任务、收集交互轨迹、基于反馈优化策略，是“从 LLM 到 Agentic RL”转变的典型代表。

核心功能

1. 该框架以部分可观测马尔可夫决策过程为基础对智能体行为进行建模，支持GRPO、RLVR等算法。
2. 其核心训练循环能够连接并调用外部工具与环境，如Web API、文件系统或浏览器模拟器，使模型能在接近真实的任务场景中学习并优化其行动策略。

用途与特点

1. 专注任务执行能力：与侧重文本对齐的框架（如TRL）不同，ART专注于培养模型的外部工具操作与任务执行能力，特别适用于构建网页浏览、代码调试、信息提取等需要与环境交互的“实干型”智能体。
2. 灵活可扩展的环境接口：提供可插拔的环境接口，允许用户灵活定义任务环境与奖励机制，从而便捷地为智能体在各种定制化任务中提供训练反馈信号。

5、Microsoft Agent-Lightning：企业级 Agentic RL 平台

Github: github.com/microsoft/ag...

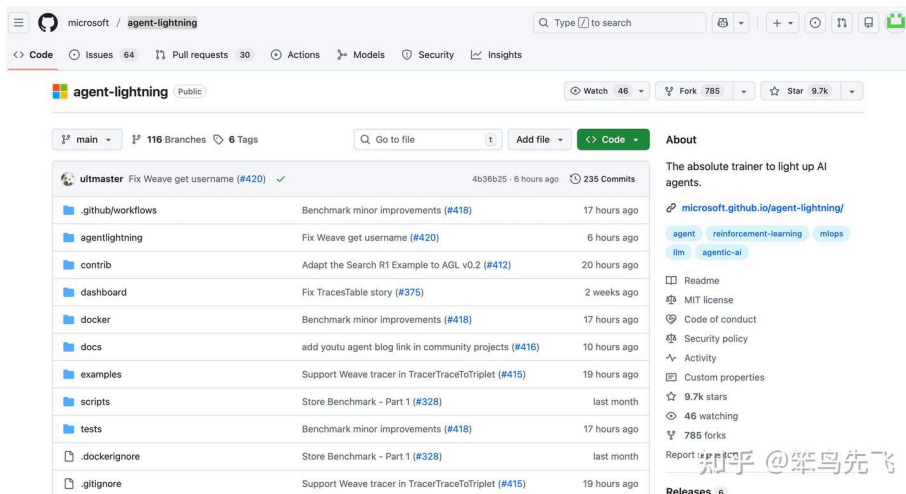


图13. Agent-Lightning库

定位：Agent-Lightning是微软于2025年推出的一个面向企业的多智能体强化学习统一平台。其设计灵感源于PyTorch Lightning的模块化理念，旨在为不同的智能体系统（如LangChain、AutoGen等）无缝嵌入标准化强化学习训练机制，提供解决方案。

1. 框架的核心由 **Manager**（编排器）、**Worker**（工作者）、**Environment**（环境）和

Orchestrator（编排器）四大模块构成。

2. 支持PPO、DPO、RLVR等多种算法，并能在多智能体协作任务中实现奖励信号的共享与自适应优化。

3. 框架内置MCP协议接口，便于灵活连接外部大语言模型服务进行协同训练。

技术特点与用途

1. 工业级多智能体协作：专为多智能体系统的复杂协作训练提供生产级支持，适用于任务自动化、AI编程助手、搜索规划等需要动态协同的场景。

2. 先进的训练与分析：具备自动奖励归因能力，并能对训练过程进行可视化分析，从而优化多智能体间的策略与协作效率。

六、Agentic-RL 业界优秀实践

本章节将介绍一些 Agentic-RL 在业界优秀实践，通过实际案例可以看出，Agentic-RL 绝非“加入几轮强化学习训练”这般简单，而是一套深度融合的“智能体行为规划 - 工具调用执行 - 实时反馈循环 - 持续线上迭代”的完整运作机制。

1、GPT-5-Codex

OpenAI明确指出GPT-5-Codex的训练流程专注于真实编程环境中的复杂任务，例如“多文件重构”“运行测试套件”与“提交PR”。这意味着，该模型并非仅仅生成一次性回答，而是扮演一个真正的编程智能体，主动贯穿“编写代码→运行代码→修正代码→提交代码”的完整 workflow。其中体现出几个关键特征：

- 以任务完成为导向：模型的训练目标不仅是输出中间答案，更强调“持续执行直至任务完成”，并以系统验证（如测试通过）作为成功的最终标志。
- 深度集成的工具调用能力：能够调用IDE接口、版本控制系统、运行时调试器等工具，这正是 Agentic RL 中“代理行为”核心能力的具体体现。
- 闭环学习与持续优化：在任务执行过程中实时接收反馈、进行纠错，并基于此类交互不断迭代与自我提升。

由此可见，Agentic RL在该系统中的实践，实质上将模型转化为一个能够自主行动、持续适应环境的“活”智能体，而不再是一个仅被动应答的聊天机器人。

2、Tongyi DeepResearch

在阿里通义的技术报告中，Tongyi DeepResearch 明确强调其模型“专为智能体任务训练”而设计：训练流程涵盖了“Agentic 持续预训练”与“冷启动 SFT + 同策略 RL”等关键环节。具体而言，该模型适配了以下典型的智能体能力：

- 长周期、跨任务的研究型行为：可执行检索、多源知识融合、自动报告生成等一系列研究动作，实现端到端的研究任务闭环。
- 深度融合的工具调用与交互：不仅限于文本生成，更能主动调用网络检索、知识库查询、结构化报告生成等外部工具与环境交互。
- 反馈驱动的训练与策略优化：在监督微调基础上引入强化学习机制，使模型能够基于任务执行反馈持续优化其决策策略。

3、Cursor 2.0 Composer

在Cursor官方介绍中，Cursor 2.0 Composer被定义为一种“AI原生的代码编辑环境”，其设计目标是将自然语言直接转化为可运行、可维护的完整项目。该工具以持续的、交互式的代码生成与编辑为闭环，融合了意图理解、代码生成与动态调试等多个环节，具体体现为以下特征：

- 父子式、迭代式的编程工作流：生成代码后仍可与用户进行多轮对话，又可根据反馈进行实时调整、重构与优化，形成“描述-生成-修改”的持续协作循环。
- 反馈驱动的代码演化机制：在编写过程中不断根据用户意图、语法检查与运行结果调整输出，逐步贴近真实工程需求，体现出学习型智能体的适应能力。

由此可见，Cursor 2.0 Composer同样内嵌了以Agentic RL为代表的智能体技术范式，将代码编辑器从“辅助工具”转变为能够理解意图、主动构建并持续演进的编程协作者，进一步强化了Agentic RL在专业工具智能化进程中作为基础架构的普遍性。

综上所述，当前顶尖的智能体系统普遍将 Agentic RL 作为核心实现路径，旨在赋予模型自主执行与环境持续协同进化的能力，而非仅依赖静态的模型微调。这也进一步印证了“Agentic RL 已成为智能体时代的标配技术”这一趋势判断。

参考文献

[1] Sutton R S, Barto A G. Reinforcement learning: An introduction 2nd ed[J]. MIT press Cambridge, 2018, 1(2): 25.

[2] Williams R J. Simple statistical gradient-following algorithms for connectionist reinforcement learning[J]. Machine learning, 1992, 8(3): 229-256.

[3] Christiano P F, Leike J, Brown T, et al. Deep reinforcement learning from human preferences[J]. Advances in neural information processing systems, 2017, 30.

[4] Schulman J, Wolski F, Dhariwal P, et al. Proximal policy optimization algorithms[J]. arXiv preprint arXiv:1707.06347, 2017.

[5] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in neural information processing systems, 2022, 35: 27730-27744.

[6] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback[J]. Advances in neural information processing systems, 2022, 35: 27730-27744.

[7] Rafailov R, Sharma A, Mitchell E, et al. Direct preference optimization: Your language model is secretly a reward model[J]. Advances in neural information processing systems, 2023, 36: 53728-53741.

[8] Shao Z, Wang P, Zhu Q, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models[J]. arXiv preprint arXiv:2402.03300, 2024.

[9] Yao S, Zhao J, Yu D, et al. React: Synergizing reasoning and acting in language models[C]//The eleventh international conference on learning representations. 2022.

[10] Zhang G, Geng H, Yu X, et al. The landscape of agentic reinforcement learning for llms: A survey[J]. arXiv preprint arXiv:2509.02547, 2025.

[11] 图解强化学习.

[12] Agentic RL详解：打造自主学习自主迭代的高性能 Agent.

所属专栏 · 2026-02-21 15:11 更新



Agentic技术

笨鸟先飞

19 篇内容 · 1238 赞同

订阅

编辑于 2025-12-18 22:30 · 上海

强化学习 (Reinforcement Learning) Agentic_ai AI大模型

聚力AI赋能
天翼云大模型Token Plan
低至9.9元

色彩安全，安全保障，多模型兼容，Token一站式调用

立即订购

天翼云Token Plan低至9.9元，助力代码开发
接入主流大模型，支持多款AI工具，按需定制，灵活采购，弹性扩容，新用户享多重专属优惠！[查看详情](#)

天翼云 的广告



理性发言，友善互动

14 条评论

默认 最新



人生还要往前走

LLM RL 优化的是「一次性吐答案」的质量；而Agentic RL 优化的是「多步交互过程」本身。



2025-12-31 · 上海

回复 2



AI2060

第三行"环境给奖励"的公式写成了： $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$ 这和第二行"环境转移"一模一样，显然是错的。奖励那一行应该写的是奖励 r_{t+1} 的生成， $r_{t+1} \sim R(\cdot \mid s_t, a_t)$

04-20 · 浙江

回复 喜欢



AI2060

最大化「期望回报」后面那个公式可以用这个 $J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [G_0] = \mathbb{E}_{\tau \sim \pi_{\theta}} [\sum_{t=0}^{\infty} \gamma^t r_{t+1}]$

04-20 · 浙江

回复 喜欢



pioneer

mark

03-16 · 上海

回复 喜欢



jack

这里的环境转移与环境给奖励一样！

1. 问题描述：马尔可夫决策过程 (MDP) 目标 ~
在策略 π 下，一次从开始到结束的交互形成一条轨迹 (trajectory)：
$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, s_2, \dots)$$

生成过程如下：
1. 初始： $s_0 \sim p_0$
2. 每一步：
• 策略选择动作： $a_t \sim \pi(a_t \mid s_t)$
• 环境转移： $s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$
• 环境给奖励： $r_{t+1} \sim R(r_{t+1} \mid s_t, a_t)$
这条轨迹的概率：
$$p_{\pi}(\tau) = p_0(s_0) \prod_{t=0}^{\infty} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t)$$

回报 Return G 定义为从当前时刻起所有的「总收益」，比如
$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

02-03 · 北京

回复 喜欢



jack

物理世界中，很多真实问题本质上就是「序列决策」，凡是符合以下几种情境的场景下，强化学习都天然适用：

后面是不是缺失文字？

02-03 · 北京

回复 喜欢



irrising

很有帮助！期待更新🙏

挺全的👍👍👍

2025-12-21 · 湖北

回复 喜欢

 笨鸟先飞 ^{作者}

谢谢~后面准备再总结下agentic rl的代码实践以及大规模训练时的踩坑经验

2025-12-22 · 上海

回复 3 喜欢

 研思索

实际中，使用开源的库就是最好的选择，问我为什么。哎，我现在就是这么干的🤔

2025-12-20 · 河北

回复 喜欢

 笨鸟先飞 ^{作者}

基本都是这样🗣️~能用开源的没必要重复造轮子，除非是场景不满足或者工作需要

2025-12-20 · 浙江

回复 喜欢

 夏焱

好长，可以拿一个下午爽看了👀

2025-12-20 · 江西

回复 喜欢

 笨鸟先飞 ^{作者}

多多交流~

2025-12-20 · 浙江

回复 喜欢

 华彩兮若英

明显是ChatGPT生成的东西直接贴过来。

03-03 · 北京

回复 1 喜欢

理性发言，友善互动

推荐阅读

统一视角看待RL从LLM到MLLM再到Agentic AI空间...

众所周知，强化学习·RL范式尝试为LLMs应用于广泛的Agentic AI甚至构建AGI打开了一扇“深度推理”的大门，而RL是否是唯一且work的一扇门，先按下不表（不作为今天跟大家唠的重点），至少目...

明了个明

Agentic RL 实战：从入门到实战！没废话，干中学！

写在前面这份指南是为想要系统学习Agentic RL（基于强化学习的智能体训练）的初学者准备的。无论你是刚接触大语言模型，还是对强化学习有一定了解但想深入Agentic领域，这份文档都会用最通...

算法全栈之路



FFM及DeepFFM模型在推荐系统的探索

张俊林

发表于深度学习前...

Agentic RL 的新范式：在强化学习循环中融合规则、模型奖...

论文标题：Exploring Reasoning Reward Model for Agents论文链接：

<https://arxiv.org/pdf/2601.22154v1>今天解读一篇来自 CUHK 与美团联合发布的论文《Exploring...

0xC001